

M04 / LEARNER BOOK

Python foundations

Standalone Zero-to-Job-Ready Data Engineer 2026

16 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE04-01 | Python setup and first program 4

DE04-02 | Variables, data types and operators 7

DE04-03 | Strings and formatting 10

DE04-04 | Conditions and boolean logic 13

DE04-05 | Lists, tuples, sets and dictionaries 16

DE04-06 | Loops and iteration 19

DE04-07 | Functions and scope 22

DE04-08 | Modules, imports and project structure 25

DE04-09 | Exceptions and debugging basics 28

DE04-10 | Files, paths, CSV and JSON I/O 31

DE04-11 | pandas foundations: load, inspect, filter and aggregate 34

DE04-12 | Practical Big-O intuition 37

DE04-13 | Coding drills: lists and strings 40

DE04-14 | Coding drills: dictionaries and sets 43

DE04-15 | Stack, queue, search and sort basics 46

DE04-16 | Integrated Python checkpoint and reproducible script habits 49

START HERE

How to use this module

Open the matching practice folder from the course root: 02_M04/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

This book is written for a beginner. Work through the lessons in order. Read the new words, watch the assigned real instructor/source range when useful, perform the guided example on your laptop, then save an independent attempt before opening the explained review. Ready-made revision is included; you do not need to rewrite the book as notes.

Study directly from the learner book and practice files. Keep your own attempts, observed outputs and result evidence. Review material is for diagnosis after a genuine attempt, not for copying. Changed retries are separate evidence. There is no fixed time-to-job promise.

Python starts here. Assume you have never programmed before. The goal is not to memorize punctuation; it is to become able to read a requirement, turn it into small Python steps, observe what the program actually did, repair failures without panic, and prove that the output is correct.

M04 stays intentionally local and lightweight. Git, Linux/WSL, Docker, Airflow and Spark come later. We add pandas only when ordinary Python file handling is already familiar. The current public Python release checked for this review build is Python 3.14.7. If a newer stable Python release appears before you study, use the current stable version from Python.org and record the version that actually installs on your PC. Do not install a preview release merely because its number is higher.

Every lesson follows one learning contract even when the headings are natural rather than numbered: understand why the skill exists, learn the new words, watch the assigned real instructor range when useful, follow one worked example, predict before running, inspect the actual result, learn at least one failure mode, do a different task alone, validate it independently, explain it back, and use the separate Explained Reviews only after saving a genuine attempt. If review was needed, close it and complete the changed retry. Your first imperfect attempt is evidence; do not overwrite it.

Do not create one giant notebook that hides which lesson produced which evidence. A simple folder with small .py files, result text and repair notes is enough. The provided study-record-template.txt can be copied for each lesson.

DE04-01

Python setup and first program

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

00:08:42-00:18:52

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Start with the machine, not with syntax

A program can be perfectly written and still fail because the wrong Python executable is running, the file was saved somewhere else, or the terminal is looking at a different folder. A data engineer eventually runs code through schedulers and servers, so the habit starts now: prove the runtime and prove the file you actually executed.

Before this lesson, all you need is the M00 computer/file foundation. No prior programming knowledge is assumed.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 00:08:42-00:18:52](#) (Getting Started). Watch the assigned range, then close the video and use this book. The public title and chapter boundary were rechecked on 14 September 2026. The course book, not the video, defines our evidence and failure exercise.

Picture what is happening

Think of a .py file as a written work order and the Python interpreter as the worker reading it from top to bottom. The file is not "running" merely because it exists. When you type a command such as:

```
py p01_first_program.py
```

you are asking a particular Python installation to open that particular file and execute its statements. The terminal then shows standard output or an error/traceback. Those are observations from a real run; the text in the editor is only the intended program.

New words you need now:

Word	Meaning here
interpreter	Program that executes Python code
script	A saved Python source file, usually ending .py
statement	An instruction Python executes
standard output	Normal text a program prints to the terminal

Word	Meaning here
syntax	Grammar rules that make Python text parseable
traceback	Error report showing where Python failed and how execution reached that point
working directory	Folder the terminal currently treats as its starting location

Follow with me once

Open a fresh terminal. First run:

```
py --version
```

Record the exact output. If `py` is unavailable but `python --version` works, record that instead; do not pretend the commands are interchangeable on every PC.

Create a file named `hello_m04.py` containing:

```
print("Standalone DE 2026")
print(7 * 8)
```

Before running it, predict the output: one line of text and then 56. Now run the file from the folder where it is saved:

```
py hello_m04.py
```

Expected output:

```
Standalone DE 2026
56
```

The multiplication happens before `print` displays the result. Nothing was stored automatically in a spreadsheet or database; output exists only because the program printed it.

Now deliberately break the first line by removing the closing quote:

```
print("Standalone DE 2026)
```

Run again. Do not immediately repair it. Read the last part of the error. Python should report a `SyntaxError` and point near the string that never closed. Save that failed terminal output. Then repair the quote and rerun.

This tiny failure exercise matters because debugging begins with evidence. If you repair before observing, you lose the chance to learn how Python describes failure.

What can go wrong on day one

If the terminal says the command is not recognized, the problem is not your `print` statement; Python/launcher discovery failed. If Python says it cannot open the file, check the working directory and filename. If a file named `hello_m04.py.txt` appears in Explorer, Windows may have hidden the extension. If your editor "runs" a file but the terminal does something different, write down the exact interpreter each environment uses rather than guessing.

A common beginner reaction is to reinstall everything. Avoid that. Isolate the boundary:

1. Can the terminal report a Python version?
2. Can the terminal show the folder containing your file?
3. Does the filename exactly match?
4. Can a two-line script run?
5. Only then investigate editor configuration.

Your independent attempt

Close the worked example. Create `p01_first_program.py` that prints your name, Standalone DE 2026, and the result of $7 * 8$. Then introduce a different syntax error from the one above, save the traceback, diagnose it in one sentence, repair it and rerun from a fresh terminal.

Do not count "the green Run button worked once" as proof. Your evidence should include the script, version command, first failure, repaired output and a short explanation of the edit -> run -> observe -> repair loop.

Changed retry after review

If you needed the Explained Review, close it before retrying. Create another file that prints a different label and calculates $9 * 6$. Introduce another syntax error deliberately, explain the final exception line before changing code, then repair it without reopening the review.

Revision in your own words

A Python source file is instructions; the interpreter executes them. Prove the interpreter with a version command, prove the file/path you ran, read the last exception line before editing, and preserve one failed attempt as debugging evidence.

DE04-02

Variables, data types and operators

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

00:18:52-00:48:41

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

A value that looks numeric may not be a number

Data engineering scripts carry many kinds of values at once: order IDs, quantities, prices, booleans, timestamps and missing fields. Treating everything that contains digits as a number is a classic way to destroy identifiers such as 0007 or attempt arithmetic on fields that should never be averaged.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 00:18:52-00:48:41](#)

(Variables, Python terms and data types) in the same course. Close the video before the independent task.

A variable is a name bound to an object/value. A type describes what kind of value Python is dealing with and therefore which operations make sense. For this lesson, know these five:

Type Example Typical meaning

int
4

whole-number quantity/count

float
850.0

measurement/price with decimal representation

str
"02001"

text or identifier

bool
True

yes/no condition

NoneType
None

no Python value supplied

Assignment is not comparison

This line binds a name to a value:

```
qty = 4
```

This line asks a yes/no question:

```
qty == 4
```

The first uses = for assignment. The second uses == for equality comparison and returns a boolean.

Follow with me once

Type this yourself:

```
order_id = "0007"
qty = 4
unit_price = 850.0
status = "Completed"
sales_rep = None

gross_sales = qty * unit_price
is_large = gross_sales >= 3000

print(order_id, type(order_id))
print(qty, type(qty))
print(unit_price, type(unit_price))
print(gross_sales, type(gross_sales))
print(is_large, type(is_large))
print(sales_rep, type(sales_rep))
```

Predict before running. order_id should remain text and keep its leading zeros. gross_sales should be 3400.0, and is_large should be True.

Why does "0007" remain a string? Because its role is identity, not quantity. Turning it into integer 7 would discard formatting and imply arithmetic meaning that the business field does not have.

Now test a failure deliberately:

```
qty_text = "4"
print(qty_text * 3)
```

This does not produce 12. String multiplication repeats text, producing 444. Python is behaving consistently with the type you gave it. If you actually received a quantity from text input and the business contract says it must be an integer, parsing is a deliberate boundary decision:

```
qty_number = int(qty_text)
print(qty_number * 3)
```

Do not turn this into a habit of blindly calling int() on every digit-like field. First decide whether the field is a measure or an identifier.

Arithmetic and comparisons

For beginner data work:

```
+ - * / // % **
```

perform arithmetic, while:

```
== != > < >= <=
```

produce booleans. A boolean can later control an if branch.

Be careful with floating-point equality. Binary floating-point cannot represent every decimal fraction exactly. For monetary production systems you will later learn stronger numeric/decimal choices. In this foundation lesson, don't build logic that assumes every calculated floating value has a perfect decimal representation.

Independent task

Create:

```
order_id = 03107
qty = 4
unit_price = 850.0
status = Completed
```

using appropriate Python types. Calculate gross sales and a boolean for gross sales at least 3000. Print every value with `type(...)`. Explain in your own words why `order_id` is text while `qty` is numeric.

Validation is not "the script ran." Check that the numeric result is 3400.0, the threshold result is true, the ID remains exactly 03107, and you can explain `=` versus `==` without notes.

Changed retry after review

Use `shipment_id="S009"`, `cartons=7`, `unit_cost=425.5`, `delivered=False`. Calculate

shipment cost and:

```
needs_review = cost > 2500 and not delivered
```

Predict both outputs before running. Then explain why `not delivered` is meaningful only because `delivered` is a boolean contract rather than a string such as `"False"`.

Revision in your own words

Choose a Python type from business meaning, not appearance. IDs are often strings even when they contain digits. `=` assigns; `==` compares. Arithmetic produces values; comparisons produce booleans. Inspect `type(value)` before "fixing" a mysterious operation.

DE04-03

Strings and formatting

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

00:48:41-01:15:10

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Text cleaning is not identity resolution

Source systems rarely send perfectly formatted labels. You may see " north HUB ", "North Hub", and "NORTH HUB". Python can normalize those strings for comparison or display, but formatting similarity does not prove that two records represent the same warehouse, customer or product.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 00:48:41-01:15:10](#) (Strings and string formatting). The video demonstrates Python string behavior; this book adds data-provenance and cleaning boundaries.

A string is a sequence of Unicode characters. Strings are immutable: operations such as `.strip()` and `.lower()` return new strings rather than editing the existing string in place.

Preserve raw and cleaned values separately

Start with:

```
raw_label = "  north HUB  "
clean_label = raw_label.strip().lower()

print(repr(raw_label))
print(repr(clean_label))
```

Expected:

```
'  north HUB  '
'north hub'
```

`repr(...)` is useful here because it makes surrounding spaces visible. If you printed the raw value normally, you might miss the leading/trailing spaces with your eyes.

Why keep both? The raw value is provenance: what the source actually supplied. The cleaned value is a transformation you chose. If the cleaning rule later proves too aggressive, the raw value lets you reproduce and revise the transformation.

Useful operations, with their limits

```
text.strip()      # surrounding whitespace
text.lower()      # lowercase
text.upper()      # uppercase
text.replace(a,b) # replace occurrences
text.startswith(x)
text.endswith(x)
text.split(sep)
sep.join(parts)
```

Do not read the comments as promises about business correctness. `.replace(" ", "")` can remove meaningful spaces. `.lower()` can normalize case but cannot prove identity. `split()` can parse a simple delimiter but is not a CSV parser when quoting/escaped delimiters matter.

Build a message with an f-string

An f-string makes values visible inside a human-readable string:

```
shipment_id = "S009"
cartons = 7
unit_cost = 425.5
cost = cartons * unit_cost

message = f"Shipment {shipment_id}: {cartons} cartons, cost={cost:.2f}"
print(message)
```

Expected cost is 2978.50. `:.2f` controls display formatting. It does not change the business meaning of the original numeric value into "money-safe decimal arithmetic"; it only controls how this value is formatted in the string.

Indexing and slicing: useful, but don't infer contracts from samples

Python strings can be indexed and sliced:

```
code = "WH-EAST-007"
print(code[0:2])
print(code[-3:])
```

You might get WH and 007. That does not mean all future IDs are guaranteed to have the same shape. Before slicing production identifiers, obtain or define the source contract.

Deliberate collision example

Run:

```
raw_values = [" Pen ", "PEN", "pen", "Pen Blue"]
for value in raw_values:
    print(repr(value), "->", value.strip().lower())
```

Three values collapse to pen; one becomes pen blue. If the first three are known variants of the same product label, that may be useful. If they are case-sensitive external keys, collapsing them would be corruption. Cleaning is a rule, not a universal truth.

Independent task

Start with:

```
raw_label = "  NORTH Hub  "
```

```
shipment_id = "s009"
```

Keep the raw label. Create a cleaned display label using deliberate `.strip()` and case normalization. Create an uppercase display form of the shipment ID without overwriting the original. Build an f-string explaining raw -> cleaned values. Then demonstrate one pair of strings that look similar after formatting but should not automatically be treated as the same identity.

Validate by printing `repr(raw_label)`, the cleaned value and the message. Explain why a formatted value can be useful for display while an authoritative key should come from the data contract.

Changed retry after review

Given " Pen Blue ", "pen blue" (two internal spaces) and "PEN BLUE", normalize only surrounding spaces and case. Compare results. Do not silently collapse internal spaces. Explain what additional business rule would be needed before deciding those labels are equivalent.

Revision in your own words

Strings are immutable. Preserve raw text, create cleaned/display values separately, and use `repr()` when invisible whitespace matters. An f-string formats a message; formatting is not identity proof. Never turn "looks similar" into "same entity" without a declared rule.

DE04-04

Conditions and boolean logic

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

01:26:20-01:34:24

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Business rules become branches

A pipeline repeatedly asks questions: Is the record valid? Is the file late? Is the order eligible? Should this row go to quarantine? Python conditions are how written business rules become executable decisions.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 01:26:20-01:34:24](#) (Conditional Statements). Our lesson adds boundary testing and rule validation.

A condition is an expression interpreted as true or false. if, elif and else choose which block executes. Indentation is part of Python syntax: it marks which statements belong to the branch.

Translate the English sentence first

Business rule:

> An order is priority only when its status is Completed and its amount is at least 2500.

Write the logic before the code:

```
status == Completed
AND
amount >= 2500
```

Then Python:

```
status = "Completed"
amount = 2500

if status == "Completed" and amount >= 2500:
    label = "priority"
else:
    label = "standard"

print(label)
```

At exactly 2500 the label should be priority because the rule says at least, meaning >= rather than >.

Prove the boundary, not only a comfortable example

Use this test table before running:

Status	Amount	Expected
Completed	2499	standard
Completed	2500	priority
Completed	2501	priority
Pending	3000	standard

Run all four. The exact boundary is often where real defects hide. One happy-path test cannot prove $\geq$ was implemented correctly.

and, or, not

and requires both conditions. or accepts at least one. not reverses a boolean.

Consider:

```
status == "Completed" or amount >= 2500
```

That would incorrectly label Pending/3000 as priority. The code can run perfectly and still implement the wrong policy.

When rules mix and and or, use parentheses to make intent visible rather than relying on remembered precedence:

```
eligible = (region == "East" or region == "West") and status == "Completed"
```

Missingness is not the same as false

Suppose delivered may be True, False or None. None means "no value supplied," not automatically "not delivered." A rule such as:

```
if not delivered:
```

would treat both False and None as falsy. That may be wrong if unknown delivery status must be quarantined separately. Prefer explicit contracts:

```
if delivered is True:
    state = "delivered"
elif delivered is False:
    state = "not_delivered"
else:
    state = "unknown"
```

This is your first glimpse of a recurring engineering principle: unknown is not automatically zero/false/no.

Recovery ladder

If a condition behaves unexpectedly:

1. Rewrite the business rule in one sentence.
2. Split it into individual booleans.
3. Print each boolean separately.

4. Test below / exactly at / above every numeric boundary.
5. Include a disqualifying case.
6. Only then combine the expression.

Independent task

Implement the priority rule above from a blank file and test all four boundary cases. Save predictions before execution. Add one explicit None example for a boolean-like field and explain why unknown should not automatically equal false.

Changed retry after review

A shipment is escalate only when `delayed_days >= 2` and `priority_customer` is `True`. Test 1, 2 and 3 delayed days for both customer flags, plus one `priority_customer=None` case. State what your code does with unknown priority and why.

Revision in your own words

Write the English rule first. `and` means all required conditions; `or` means at least one; parentheses make mixed rules auditable. Test boundaries and disqualifying cases. None can mean unknown and should not be silently treated as a business False unless the contract says so.

DE04-05

Lists, tuples, sets and dictionaries

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

01:34:24-02:12:26

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Four containers, four different promises

Python gives you several built-in collection types. Beginners often learn them as syntax to memorize; an engineer chooses them based on the promise the program needs: preserve sequence, keep a fixed grouping, enforce uniqueness/membership, or map a key to a value.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 01:34:24-02:12:26](#) (Lists, dictionaries, sets and tuples). Watch the demonstrations, then return here to choose by engineering requirement rather than visual preference.

Picture four containers:

- a list is an ordered tray whose contents can change;
- a tuple is an ordered grouping commonly used when the grouping should be treated as fixed;
- a set is a membership collection that keeps unique hashable values and does not promise
- list-style positional order;
- a dictionary maps unique keys to values.

Start with the requirement, then choose

```
pipeline_steps = ["extract", "validate", "publish"]
period = (2026, 9)
failed_batch_ids = {"B17", "B22", "B17"}
region_manager = {"East": "Asha", "West": "Om"}
```

```
print(pipeline_steps)
print(period)
print(failed_batch_ids)
print(region_manager["East"])
```

`pipeline_steps` is a list because sequence matters and we may add/remove a step. `period` is a tuple because year/month belong together and we do not need in-place edits. The set keeps B17 once because membership/uniqueness is the purpose. The dict lets us retrieve a manager by region key.

Mutation matters

Lists and dictionaries are mutable. That means this:

```
steps = ["extract", "load"]
other_name = steps
```

```
other_name.append("validate")
print(steps)
```

prints a list that now includes validate. `other_name` did not create an independent copy; both names refer to the same list object.

For a shallow copy of a flat list:

```
copied = steps.copy()
```

creates another outer list. Later you will learn that nested mutable objects make copying more subtle; do not build a false rule that `.copy()` recursively duplicates everything.

This distinction matters in pipelines because one helper can unexpectedly mutate data another part of the program still expects to be unchanged.

Sets solve membership problems, not ordering problems

```
seen = set()
for order_id in ["01", "02", "01"]:
    if order_id in seen:
        print("duplicate candidate:", order_id)
    else:
        seen.add(order_id)
```

The set is useful because we care whether an ID has appeared, not where it appears by index. A set can be faster for repeated membership checks than scanning a list; DE04-12 explains that growth intuition.

But if first-occurrence order matters, a set alone is insufficient. One common pattern is a set for membership plus a list for ordered output:

```
seen = set()
unique_in_order = []
for order_id in ["02", "01", "02", "03"]:
    if order_id not in seen:
        seen.add(order_id)
        unique_in_order.append(order_id)
```

Dictionary keys are an identity contract

A dict cannot hold two simultaneous values under the same key in the same mapping. Assigning the key again replaces the previous value:

```
manager = {"East": "Asha"}
manager["East"] = "Ravi"
```

That is not a duplicate-record store. If multiple managers per region are valid, the value itself could be a list, or you may need a different data model. The collection should match the relationship.

Use `.get()` when absence is expected and you have consciously chosen how to represent it:

```
print(manager.get("North"))          # None
print(manager.get("North", "Unknown"))
```

Be careful: the display label Unknown is your added fallback. It is not a region supplied by the source.

Independent task

Represent four requirements:

1. three processing steps that must keep order;
2. a fixed (year, month) reporting period;
3. failed batch IDs where duplicates should collapse;
4. a region-to-manager lookup.

Print and manipulate each structure enough to prove its behavior. Then create a list, assign a second name to it, mutate through the second name, and explain why the original changed. Create a separate copy and prove the outer list is independent.

Changed retry after review

Represent warehouse zones and supervisors, a unique set of failed shipment IDs, a processing sequence, and a fixed (country_code, year) pair. Add a case where one zone legitimately has two supervisors and redesign the dictionary value rather than silently overwriting one name.

Revision in your own words

List = ordered mutable sequence. Tuple = ordered fixed-style grouping. Set = unique membership. Dict = key -> value lookup. Choose by the requirement, not by whichever syntax you remember. Assignment can create two names for the same mutable object; copying behavior must be deliberate.

DE04-06

Loops and iteration

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

02:21:06-02:44:16

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Repetition should make work clearer, not hide repeated mistakes

A data pipeline applies logic to many records. Loops let you express "for each item" without copy-pasting the same statements. The risk is equally important: if the logic is wrong, a loop repeats the wrong action consistently across the dataset.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 02:21:06-02:44:16](#) (Loops and list-comprehension lead-in). The course uses explicit loops first because they make state changes visible.

An iterable is an object you can step through. An iteration is one pass through the loop body. An accumulator is a value updated across iterations, such as a running total. `break` exits a loop early; `continue` skips to the next iteration.

Follow one record at a time

```
amounts = [1200, 500, 0, 800, 1500]
total = 0
positive_count = 0
invalid = []
```

```
for amount in amounts:
    if amount <= 0:
        invalid.append(amount)
        continue
    total += amount
    positive_count += 1
```

```
print(total)
print(positive_count)
print(invalid)
```

Predict first. Positive total should be 4000, positive count 4, invalid list [0].

The source list is unchanged. We created result state separately. That is a useful default habit: do not mutate the collection you are currently iterating over unless you can prove the behavior you want.

Why mutating while iterating can surprise you

```
values = [1, 0, 0, 2]
for value in values:
    if value == 0:
        values.remove(value)
print(values)
```

A beginner may expect `[1, 2]`, but removing elements shifts positions while the iterator advances. One zero can be skipped. Prefer a new collection:

```
clean = [value for value in values if value != 0]
```

or an explicit loop appending accepted values. The important principle is not "list comprehensions are always better"; it is do not make source mutation an accidental side effect of traversal.

range and enumerate

Use range when you genuinely need a sequence of integers. Use enumerate when you need both position and value:

```
steps = ["extract", "validate", "publish"]
for position, step in enumerate(steps, start=1):
    print(position, step)
```

Do not use indexes merely because you learned them first. `for row in rows:` is usually clearer than `for i in range(len(rows)):` when you only need each row.

Loop over dictionaries deliberately

```
regional_sales = {"East": 11700, "West": 8100}
for region, sales in regional_sales.items():
    print(region, sales)
```

`.items()` makes key and value explicit. Iterating directly over the dict yields keys. Neither is "the right way" universally; choose the shape your loop needs.

Nested loops and cost intuition

If every left record scans every right record, work grows quickly:

```
for order in orders:
    for customer in customers:
        ...
```

You do not need full algorithm analysis yet, but notice the pattern. DE04-12 will connect it to $O(n^2)$ -style growth and lookup structures.

Independent task

Using `amounts=[1200,500,0,800,1500]`, calculate positive total/count and collect invalid non-positive values without changing the original list. Prove the original input is unchanged. Then iterate a dictionary of three regional totals and print a numbered report using enumerate over a deliberately ordered list of regions.

Add a test with an empty `amounts=[]`. Decide what total/count should be and prove your code handles it without a special crash.

Changed retry after review

Given shipment weights `[12.5, -1, 0, 8.0, 15.2]`, create accepted and rejected lists, total only positive weights, and leave the source list intact. Explain why removing bad values from the same list during iteration is a weaker design.

Revision in your own words

A loop repeats a block for each item. Keep input and result state separate when possible. Use `continue` to skip a record deliberately, `break` only when early termination is truly the requirement, `enumerate` for position + value, and `.items()` for dict key/value pairs. Do not casually mutate the collection you are traversing.

DE04-07

Functions and scope

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

02:52:12-03:12:46

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

A pipeline is easier to trust when logic has named boundaries

Copy-pasted calculations are hard to test and easy to let drift. A function gives a name to a piece of behavior, defines what inputs it receives and what output it returns. Later modules will build ingestion and transformation pipelines from functions, so the key habit is to make dependencies visible.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 02:52:12-03:12:46](#)

(Functions; lambda is optional context). Focus on function definition/calls/return values. Lambda syntax is not a mastery requirement here.

New words:

- function: reusable block of behavior called by name;
- parameter: name in the function definition;
- argument: actual value supplied in a call;
- return value: value sent back to the caller;
- local variable: name created inside the function's local scope;
- scope: rules determining where a name can be accessed;
- pure-ish function: informal term here for a function whose result mostly depends on explicit inputs and avoids hidden side effects.

Follow with me once

```
def summarize_order(order_id, qty, unit_price):
    gross_sales = qty * unit_price
    return {
        "order_id": order_id,
        "gross_sales": gross_sales,
    }

first = summarize_order("01", 2, 850.0)
second = summarize_order("02", 3, 500.0)
print(first)
print(second)
```

Expected gross values are 1700.0 and 1500.0. The function does not know about a global qty; it receives all three required values through parameters.

print is not return

This function:

```
def show_total(qty, unit_price):  
    print(qty * unit_price)
```

displays a value but implicitly returns None. If later code needs the number for validation, filtering or aggregation, printing is not enough.

```
def calculate_total(qty, unit_price):  
    return qty * unit_price
```

Now the caller can store, test or combine the result.

Hidden global state makes behavior harder to reason about

```
tax_rate = 0.18
```

```
def total_with_tax(amount):  
    return amount * (1 + tax_rate)
```

This can work, but the function depends on a name not visible in its signature. If `tax_rate` changes elsewhere, the same call may produce a different answer. Configuration globals are sometimes legitimate, but understand the dependency. For a beginner exercise, explicit input is clearer:

```
def total_with_tax(amount, tax_rate):  
    return amount * (1 + tax_rate)
```

Default arguments and mutable defaults

A safe simple default:

```
def label_order(order_id, prefix="ORDER"):  
    return f"{prefix}:{order_id}"
```

A classic pitfall:

```
def add_error(message, errors=[]):  
    errors.append(message)  
    return errors
```

That list is created once when the function is defined and reused across calls. Prefer:

```
def add_error(message, errors=None):  
    if errors is None:  
        errors = []  
    errors.append(message)  
    return errors
```

You do not need to memorize every Python function nuance now; you do need to learn that hidden shared mutable state can leak across calls.

Validate functions with multiple inputs

One successful call proves little. For `summarize_order`, try:

- normal positive quantity;
- zero quantity;

- another price;
- later, an invalid type to see the failure boundary.

A function contract should state what valid inputs mean and what output shape it returns.

Independent task

Write `summarize_order(order_id, qty, unit_price)` returning a dict with `order_id` and

`gross_sales`. Test two normal inputs and one zero-quantity input. Show that the returned dict can be assigned and reused. Then write a deliberately weak version that prints but returns nothing, inspect its actual return with `print(type(result))`, and explain the difference.

Changed retry after review

Write `shipment_cost(shipment_id, cartons, cost_per_carton)` returning a dict with ID and cost. Test three cases including zero cartons. Then extend it to accept an explicit currency argument rather than reading a hidden global. Explain which inputs are measures and which are identifiers/labels.

Revision in your own words

Functions create named behavior boundaries. Parameters make dependencies visible; arguments are supplied values; return gives a result to the caller. Printing is observation, not a reusable return value. Avoid hidden mutable state and test more than one input.

DE04-08

Modules, imports and project structure

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

03:12:46-03:40:05

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

One file is useful until it becomes one giant file

As programs grow, keeping every helper and every execution step in one script becomes difficult to navigate and test. A Python module is simply a Python file that can be imported. The key beginner lesson is not "make lots of folders"; it is to separate reusable logic from the code that starts a run, while understanding that importing a module executes its top-level statements once per interpreter process.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 03:12:46-03:40:05](#)

(Modules and Python library lead-in). Our book adds import side-effect and entry-point reasoning.

Use the supplied `practice/stage04_helpers.py`:

```
def gross_sales(qty, unit_price):  
    return qty * unit_price
```

```
def normalized_label(text):  
    return text.strip().lower()
```

Create a separate `run_order.py` in your working folder:

```
from stage04_helpers import gross_sales, normalized_label
```

```
def main():  
    amount = gross_sales(3, 500.0)  
    label = normalized_label("  EAST HUB ")  
    print(label, amount)
```

```
if __name__ == "__main__":  
    main()
```

Expected output is east hub 1500.0.

What `__name__` is protecting

When a file is run directly, Python sets its `__name__` to `"__main__"`. When it is imported, `__name__` is the module name. Therefore the guard:

```
if __name__ == "__main__":  
    main()
```

lets you keep "start the program" behavior separate from "make functions available for import."

Deliberately add this line at the top level of the helper file:

```
print("helper imported")
```

Then run `run_order.py`. You will see the message during import. That proves top-level module code executes as part of import. Remove the diagnostic after understanding it. A module should not unexpectedly start a data pipeline merely because another file imported one helper.

Import styles

```
import stage04_helpers
stage04_helpers.gross_sales(2, 850)
```

makes the module name explicit.

```
from stage04_helpers import gross_sales
gross_sales(2, 850)
```

is shorter. Both are valid. Avoid wildcard imports such as `from module import *` in our course because they hide which names entered the current namespace.

Name collisions and shadowing

Do not name your learner file `pandas.py`, `csv.py`, `json.py` or `pathlib.py` when you intend to import the standard/library package of that name. Python can find your local file first and import the wrong thing. If an import behaves impossibly, inspect the traceback path and `module.__file__` where appropriate.

Keep structure small in M04

A simple M04 shape is enough:

```
my_m04_work/
  stage04_helpers.py
  run_order.py
  evidence/
```

Do not add Git packaging, virtual-environment architecture or `src/` layout yet just to look professional. M06 teaches repository engineering. DE04-16 will show a reproducible folder/run contract, and its post-attempt source introduces modern project structure at the right time.

Independent task

Use `stage04_helpers.py` from a separate entry script. Import and call both functions. Add a temporary top-level `print` to the helper to prove imports execute top-level code, save the observed result, then remove that side effect. Add a `main()` function and `__main__` guard to the entry script. Finally import the entry script from another tiny file and prove its `main()` did not run automatically.

Changed retry after review

Create `shipment_helpers.py` with `shipment_value(cartons, unit_cost)` and

`normalize_hub(text)`. Create `run_shipment.py` with `main()`. Prove the helper can be imported without printing or starting the workflow. Explain which file owns reusable logic and which owns run orchestration.

Revision in your own words

A module is a Python file that can be imported. Import executes top-level module code, so keep side effects deliberate. Use a `main()` function plus `if __name__ == "__main__":` for an explicit entry point. Avoid wildcard imports and local filenames that shadow standard/package modules.

DE04-09

Exceptions and debugging basics

Watch first

[1. Corey Schafer - Try/Except Blocks](#)

OPTIONAL visual - no verified bounded range. Use only for try/except demonstration; learner book + practice are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

The traceback is evidence, not an insult

Programs fail. Data files disappear, values arrive with unexpected types, keys are missing and network calls time out. An engineer's job is not to make errors invisible; it is to understand which failures are expected, which can be handled safely, and which should stop the run loudly.

[Instructor demonstration: Corey Schafer - Python Tutorial: Using Try/Except Blocks for Error Handling](#). Our supplied M04_Debug_Me.py is the authoritative practice problem.

OPTIONAL visual - no verified bounded range.

An exception is an object representing a runtime error condition. A traceback shows the chain of calls that led to an unhandled exception. try marks code that may raise an expected exception; except handles a selected exception type. else can run when no exception occurred; finally runs whether the try succeeded or failed, commonly for cleanup.

Read the traceback from the bottom first

Open a copy of practice/M04_Debug_Me.py. It contains records with deliberately inconsistent values. Run it without editing. Save the full traceback.

A traceback often contains several file/line frames, but the final line normally tells you the exception class and message. For example:

```
TypeError: can't multiply sequence by non-int of type 'float'
```

Do not search the entire internet for that sentence before checking the values at the failing expression. Add temporary observations:

```
print(order)
print(type(order["qty"]), type(order["unit_price"]))
```

The purpose is to identify the first broken assumption. In the starter file, one quantity is text. The safe repair depends on the contract: if the input specification says quantity must be an integer represented as text, parse and validate it; if arbitrary text is allowed, the calculation itself may not be valid.

Catch narrowly, not broadly

A dangerous beginner pattern is:

```
try:
```

```
...  
except Exception:  
    pass
```

This hides every ordinary exception and throws away the reason. The program may then continue with partial or wrong output.

A more deliberate boundary might be:

```
raw_qty = "3"  
try:  
    qty = int(raw_qty)  
except ValueError as exc:  
    print(f"invalid quantity: {raw_qty!r}")  
    raise
```

Here we expect `int(...)` can raise `ValueError` for non-integer text. We log context and re-raise because this beginner script has no approved quarantine policy yet. Later modules will build rejected-row handling.

Exceptions are not a substitute for validation

If the business contract requires `qty > 0`, successful `int("0")` parsing does not make zero valid. Type conversion answers "can Python represent this as an integer?" Business validation answers "is this value allowed for this field?" Keep those questions separate.

Deliberate division failure

Consider:

```
sales = 0.0  
qty = 0  
average_unit = sales / qty
```

This raises `ZeroDivisionError`. Catching it and returning zero would invent a meaning unless the business contract explicitly defines an average for zero quantity. Sometimes the right behavior is to reject/flag the record, not force a numeric answer.

Debug one cause at a time

Use this order:

1. Reproduce the failure.
2. Save exact input and traceback.
3. Read the final exception type/message.
4. Find the earliest line where an assumption becomes false.
5. Add the smallest diagnostic observation.
6. Repair one cause.
7. Rerun the same case.
8. Remove temporary debug prints or convert them into deliberate logging later.
9. Add a test that would catch the failure if it returns.

Changing five lines at once destroys evidence about which change fixed the problem.

Independent task

Work on a copy of `M04_Debug_Me.py`. Preserve the first traceback. Fix the first type problem deliberately, then observe the next failure if one appears. For each failure, record exception type, failing expression, violated assumption and repair. Add a narrow exception boundary only where you can name the expected failure and what your program should do about it. Do not use bare `except:` or silently pass.

Changed retry after review

Write `safe_positive_int(raw_value)` that either returns a positive integer or raises a clear `ValueError` for non-integer text, zero and negative values. Test `"4"`, `"0"`, `"-2"` and `"four"`. Explain why parsing and business validation are two different checks.

Revision in your own words

Preserve the traceback. Read the last exception line, inspect values/types at the failing expression, repair one assumption at a time and rerun. Catch only exceptions you understand and can handle safely. Parsing success does not prove business validity, and hiding an error is not recovery.

DE04-10

Files, paths, CSV and JSON I/O

Watch first

[1. Corey Schafer - File Objects](#)

OPTIONAL visual - no verified bounded range. Use only for file-object context; learner book + pathlib/CSV/JSON practice are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Data engineering crosses boundaries between memory and files

So far most values disappeared when the script ended. Real pipelines read files, validate them and write outputs another process can consume. File I/O introduces new failure boundaries: path, encoding, file format, schema and serialization.

[Instructor demonstration: Corey Schafer - Python Tutorial: File Objects - Reading and Writing to Files.](#)

OPTIONAL visual - no verified bounded range; book owns file-format contracts.

New words:

- path: location of a file/folder;
- relative path: location interpreted from a known base rather than a machine-specific
- absolute drive path;
- encoding: mapping between characters and bytes; use UTF-8 explicitly in our text files;
- serialization: converting an in-memory structure to a storable/transmittable representation
- such as JSON;
- schema: expected fields/types/meaning;
- read-back validation: reopening what you wrote and checking it rather than trusting the
- write call.

Build paths from a known file location

Hard-coding:

```
path = r"C:\Users\Ashish\Desktop\orders.csv"
```

makes the script depend on one personal machine. In a file living under practice/, you can construct a sibling project path using pathlib:

```
from pathlib import Path

HERE = Path(__file__).resolve().parent
DATA = HERE.parent / "data"
orders_path = DATA / "M04_Retail_Orders.csv"
```

```
print(orders_path)
print(orders_path.exists())
```

`__file__` identifies the script file. `resolve()` gives a resolved path. Building from that location is more reproducible than hoping the current working directory is wherever you expected.

Read CSV with the standard library

```
import csv

with orders_path.open("r", encoding="utf-8", newline="") as handle:
    reader = csv.DictReader(handle)
    rows = list(reader)

print(reader.fieldnames)
print(len(rows))
print(rows[0])
```

A CSV is text. DictReader returns field values as strings. The fact that `qty` looks like "2" does not make it an integer. Parsing belongs at the boundary with explicit error handling.

Do not use `line.split(',')` as a replacement CSV parser. Quoted commas and escaping rules make CSV more than "text separated by commas."

Validate schema before doing calculations

```
required = {"order_id", "region", "status", "qty", "unit_price", "sales_rep"}
actual = set(reader.fieldnames or [])
missing = required - actual
if missing:
    raise ValueError(f"missing required columns: {sorted(missing)}")
```

This fails early at the schema boundary. It is much easier to diagnose than a mysterious missing-key error halfway through the pipeline.

JSON preserves structure, not arbitrary Python objects

```
import json

manifest = {
    "source": orders_path.name,
    "expected_rows": len(rows),
    "status": "training",
}
manifest_path = HERE / "run_manifest.json"

with manifest_path.open("w", encoding="utf-8") as handle:
    json.dump(manifest, handle, indent=2, sort_keys=True)

with manifest_path.open("r", encoding="utf-8") as handle:
    loaded = json.load(handle)

assert loaded == manifest
```

`sort_keys=True` is one small step toward deterministic text output. Not every Python object is directly JSON serializable; standard JSON values are objects/maps, arrays, strings, numbers, booleans and null. A Path or datetime typically needs explicit conversion to a serializable representation.

Preserve raw input

Never "clean" the supplied CSV by overwriting it during a learning pipeline. Treat raw input as evidence. Write derived outputs to a separate path. This habit becomes critical in ingestion and lakehouse stages.

Failure exercise

Point your code at data/M04_Bad_Orders.csv. It is deliberately missing a required column and includes malformed numeric text. The first approved failure should be the schema check, because missing qty is already enough to stop the task. Your program should not stumble into later arithmetic first.

Independent task

Use `csv.DictReader` and `pathlib` to read the retail CSV. Record header and row count. Create a JSON manifest containing source filename, expected rows and status. Write with UTF-8, read it back and assert all values. Then test the bad fixture and save the visible failure. Do not use `pandas` yet.

Changed retry after review

Create a fresh manifest for M04_Retest Shipments.csv, read its CSV header with the standard `csv` module and validate these required fields: `shipment_id`, `hub`, `status`, `cartons`, `unit_cost`, `coordinator`. Add a deliberately wrong required field name and preserve the resulting missing-column error before repairing it.

Revision in your own words

Build paths from a known project/script base instead of a personal absolute path. CSV fields begin as text; parse deliberately. Validate required columns before transformation. Use UTF-8. JSON serialization needs JSON-compatible values. Read outputs back and compare critical values. Preserve raw inputs.

DE04-11

pandas foundations: load, inspect, filter and aggregate

Watch first

[1. Luke Barousse - Python for Data Analytics](#)

04:22:56-05:08:26

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

pandas is powerful enough to make a wrong answer look polished

A DataFrame can display beautiful rows while representing the wrong population, wrong types or wrong grain. Therefore the first pandas habit is not transformation - it is inspection before transformation.

[Instructor demonstration: Luke Barousse - Python for Data Analytics, 04:22:56-05:08:26](#) (Pandas intro, inspection, cleaning and analysis). Current pandas documentation checked for this review build lists pandas 3.0.5 as the latest release. Install the current compatible pandas version when you reach this lesson and record what actually installs. pandas 3.0 introduced behavior changes such as a dedicated default string dtype, so do not write a rule that all text columns must display as object.

First install at this lesson, not earlier merely because a tutorial uses it:

```
py -m pip install pandas
py -c "import pandas as pd; print(pd.__version__)"
```

Load, then inspect

```
from pathlib import Path
import pandas as pd

HERE = Path(__file__).resolve().parent
path = HERE.parent / "data" / "M04_Retail_Orders.csv"
df = pd.read_csv(path)

print(df.shape)
print(df.columns.tolist())
print(df.dtypes)
print(df.head())
print(df.isna().sum())
```

Expected input row count is 25. Do not transform yet. State the source grain: one row per order in this training fixture. order_id should therefore be unique here.

Check that assumption:

```
print(df["order_id"].nunique(), len(df))
print(df["order_id"].duplicated().sum())
```

Calculate a measure without changing source grain

```
df["sales"] = df["qty"] * df["unit_price"]
```

Adding a calculated column does not change row count or grain. It adds one measure to each existing order row.

Filter changes population, not row meaning

```
completed = df.loc[df["status"].eq("Completed")].copy()
print(len(completed))
```

Expected Completed row count is 23. One row still means one order, but now only Completed orders are eligible.

Why `.copy()` here? We are creating a separate working DataFrame for the filtered population. It makes our intention clearer when we later add/modify derived columns. pandas view/copy semantics have evolved; the course does not teach folklore like "every slice is always a view." Make transformations explicit and validate results rather than depending on undocumented assumptions.

Aggregate changes grain

```
regional = (
    completed.groupby("region", dropna=False, as_index=False)["sales"]
    .sum()
    .sort_values("region")
)
print(regional)
```

Now one row means one region, not one order. That grain change must be stated.

Expected Completed sales controls:

Region	Completed sales
East	11700
North	9310
South	10210
West	8100

Grand total should be 39320.

Reconcile independently:

```
source_total = completed["sales"].sum()
regional_total = regional["sales"].sum()
assert source_total == regional_total == 39320
```

This is stronger than "the table looks reasonable."

Missing values need a declared population

The fixture has one missing `sales_rep` value. Count it explicitly:

```
missing_rep = df["sales_rep"].isna().sum()
print(missing_rep)
```

With standard `read_csv`, the blank field is normally recognized as missing. Do not immediately `fillna("Unknown")` just to remove blanks. A display label can be useful, but missing source identity and an actual rep named "Unknown" are different provenance states.

Top-N must have deterministic sorting rules

```
top3 = (
    completed.sort_values(["sales", "order_id"], ascending=[False, True])
    .loc[:, ["order_id", "region", "sales"]]
    .head(3)
)
```

`head(3)` by itself does not mean "top 3." The sort defines "top." A secondary key makes ties deterministic.

Independent task

Produce four outputs from the retail fixture:

1. Completed sales by region;
2. top 3 Completed orders by sales with deterministic tie handling;
3. rows where `sales_rep` is missing;
4. a one-line statement of grain for each output.

Before accepting the result, prove: input rows 25; Completed rows 23; Completed sales 39320; missing sales rep 1; regional totals reconcile to the same 39320. Preserve the raw CSV unchanged.

Changed retry after review

Use `M04_RetestShipments.csv`. Calculate `shipment_value = cartons * unit_cost`; filter

Delivered; aggregate Delivered value by hub; show top 3 Delivered shipments with a deterministic second key; count missing coordinator values over the population you explicitly name. Do not look at the review controls until your query/code is saved.

Revision in your own words

Inspect shape, columns, dtypes, keys and missingness before transforming. Adding a column usually preserves grain; filtering changes population; `groupby` changes grain. State each output grain and reconcile grouped totals to a detail-level control. Top-N requires an explicit sort rule. Missing values are not automatically a label you should fill.

DE04-12

Practical Big-O intuition

Watch first

[1. freeCodeCamp - Data Structures and Algorithms in Python](#)

00:50:52-01:24:57 - Complexity and Big O notation

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Think about growth before data becomes large

A script can be correct on five rows and painfully slow on five million. Big-O notation gives a rough language for how work grows with input size. It does not predict exact seconds and it does not replace measurement/profiling.

[Instructor demonstration: freeCodeCamp - Data Structures and Algorithms in Python,](#)

00:50:52-01:24:57 (Complexity and Big O notation). This is the complete bounded Big-O chapter used by M04; stop at the Binary Search vs Linear Search chapter boundary. We are not turning M04 into a competitive-programming course.

The four growth ideas you need now

- $O(1)$: roughly constant work independent of collection size for the operation considered;
- $O(n)$: work grows proportionally with the number of items;
- $O(n \log n)$: common comparison-sort growth class;
- $O(n^2)$: quadratic growth, often from a full nested scan.

Big-O ignores constant factors and lower-order terms to focus on scaling behavior. $3n + 10$ is $O(n)$, not because 3 and 10 do not cost time, but because linear growth dominates as n gets large.

Repeated membership: list versus set

```
ids_list = ["A", "B", "C", "D"]
ids_set = set(ids_list)

print("C" in ids_list)
print("C" in ids_set)
```

A list membership check may scan values until a match is found, so worst-case growth is $O(n)$. Set membership is average $O(1)$ because sets use hashing, while acknowledging pathological/worst-case implementation details are more complex than this foundation model.

Now imagine checking 100,000 incoming IDs against 100,000 known IDs. Repeated list scans can approach a nested-work pattern; building a set once can turn the repeated membership operation into far cheaper average lookup.

Nested scan

```
for order in orders:
    for customer in customers:
        if order["customer_id"] == customer["customer_id"]:
            ...
```

If both collections grow around n and each order scans every customer, comparisons grow roughly $n \times n = n^2$. A dictionary keyed by customer ID can convert the repeated lookup into average $O(1)$ after one $O(n)$ -style build pass.

This is the same engineering instinct you used in SQL joins: identity/key structure matters.

Sorting is not free

Python's built-in sorting is highly optimized and generally has $O(n \log n)$ comparison-based behavior. That does not mean you should write your own sorting algorithm for production. The lesson is to recognize that sorting large datasets costs more than one simple scan and should serve a clear requirement.

Don't misuse micro-benchmarks

Timing four values once and seeing a set slower than a list does not disprove the growth model. Creating the set itself has cost; cache effects and tiny constants dominate small inputs. Big-O asks how the algorithm's work grows, while profiling asks how this implementation performs under realistic conditions. You will use both ideas later.

Independent task

Write comments/code that explain:

1. one full scan through n records $\rightarrow O(n)$;
2. nested full scan of n by $n \rightarrow O(n^2)$;
3. repeated membership with a set $\rightarrow$ average $O(1)$ per membership after set construction;
4. sorting $\rightarrow$ typically $O(n \log n)$ growth.

Then transform this weak idea:

```
for each shipment, scan every failed ID in a list
```

into "build a set of failed IDs once, then test membership." Explain setup cost and repeated-check cost separately.

Changed retry after review

Suppose each order must attach a region label from a customer mapping. Compare scanning every customer for each order with building one `customer_id` $\rightarrow$ region dictionary first. State approximate build cost, lookup cost and total-growth intuition. Do not claim dictionary lookup is "always literally one CPU operation."

Revision in your own words

Big-O describes growth, not exact seconds. One pass is $O(n)$; nested full scans can be $O(n^2)$; set/dict membership is average $O(1)$; sorting is commonly $O(n \log n)$. Use the model to notice repeated work, then profile real programs when performance matters.

DE04-13

Coding drills: lists and strings

Watch first

[1. Data structures and algorithms in Python - beginner course](#)

Optional visual - no verified bounded range. Use the lists, strings and dictionary demonstrations as preparation. Save your independent checkpoint attempt before returning to worked solutions.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Retrieval practice starts here

You have already seen strings, lists, loops and functions. An interview or production debugging moment will not present the lesson page beside you. This lesson begins the lightweight coding-retrieval track: small problems, closed-book first attempt, explicit input/output contract, edge cases, and explanation.

There is no external video by design. Keep videos, review answers and AI-generated finished solutions closed until your attempt is saved. The point is to retrieve skills, not recognize them while watching someone else.

Use a simple problem-solving loop

Before typing code, write four lines:

Input:
Output:
Example:
Edge cases:

Then choose a small algorithm and trace one example by hand. Only then code.

Guided pattern: reverse word order

Requirement: given a string, return the words in reverse order while preserving characters inside each word.

For:

data engineering is fun

expected:

fun is engineering data

One clear implementation:

```
def reverse_words(text):  
    words = text.split()  
    words.reverse()  
    return " ".join(words)
```

Trace it:

```
"data engineering is fun"
-> ["data", "engineering", "is", "fun"]
-> ["fun", "is", "engineering", "data"]
-> "fun is engineering data"
```

For blank input, `split()` returns an empty list and joining it returns an empty string. That is a useful edge case to test.

There are alternative valid implementations, such as slicing `words[::-1]`. Mastery is not matching one preferred line. You should be able to state the contract, predict output and defend why your code satisfies it.

Guided pattern: count vowels

```
def count_vowels(text):
    vowels = {"a", "e", "i", "o", "u"}
    count = 0
    for char in text.lower():
        if char in vowels:
            count += 1
    return count
```

For "Data", expected count is 2. `text.lower()` creates a normalized comparison view without changing the caller's original string. The set expresses membership of allowed vowel characters.

What does this function *not* claim? It does not implement every linguistic definition of vowel across all languages. The contract is explicitly the five English letters in this training drill.

Common coding-screen traps

A short problem can fail because you:

- start coding before agreeing on the output;
- ignore blank/single-item inputs;
- mutate an input when the requirement did not permit it;
- return `None` because you printed instead of returned;
- solve only the given example rather than the general pattern;
- write a clever one-liner you cannot explain.

Independent task

Open `practice/M04_Coding_Drills.py`. Implement `reverse_words` and `count_vowels` without

review open. Run `M04_Visible_Tests.py`. Then invent at least one edge case for each function beyond the supplied tests and record expected output before running.

Do not edit the visible tests just to make a failing function "pass." If you believe the test contract is wrong, explain why and preserve that disagreement as evidence.

Changed retry after review

In a fresh file, implement:

```
normalize_words(text)
```

It should strip surrounding whitespace, split on runs of whitespace, lowercase tokens and return a list of tokens. Test blank text, one word and text with multiple spaces. This is deliberately different from `reverse_words`: the output type changes and no reversing is requested.

Continuous interview habit

From this lesson onward, add one 15-20 minute closed-book Python micro-drill to study days. Stop at the time limit, preserve the attempt, review the reasoning, then redo a changed problem later. Do not spend hours on advanced competitive programming; the aim is reliable basic problem solving for DE screening and debugging.

Revision in your own words

State input/output/edge cases before code. Trace one example. Return values rather than only printing. Test blank and boundary inputs. A short, explainable solution with correct contract is stronger than a memorized clever one-liner.

DE04-14

Coding drills: dictionaries and sets

Watch first

[1. Data structures and algorithms in Python - beginner course](#)

Optional visual - no verified bounded range. Use the lists, strings and dictionary demonstrations as preparation. Save your independent checkpoint attempt before returning to worked solutions.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Use a mapping when the problem is about counts or lookup

Dictionaries and sets are especially useful in coding screens because they let you express frequency, membership and "seen before" logic clearly. In data engineering the same patterns appear in validation, deduplication candidates, lookup enrichment and state tracking.

Again, no external video by design. This is retrieval practice.

Frequency map

Requirement: count occurrences of each item while preserving the item as the dictionary key.

```
def frequency_map(items):
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts
```

For:

```
["A", "B", "A", "C", "A"]
```

expect:

```
{"A": 3, "B": 1, "C": 1}
```

Trace A: absent -> default 0 -> 1; later 1 -> 2; later 2 -> 3. The dictionary is storing one aggregate state per key.

Do not confuse this with source-grain preservation. Once you turn five events into three dictionary keys, you have aggregated occurrences by item. If each occurrence needs separate metadata, a simple frequency dictionary is no longer the whole data model.

Unique values while preserving first occurrence order

A set alone gives membership but not the list-style "first occurrence order" result contract.

Combine structures:

```
def unique_in_order(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
```

```
        result.append(item)
    return result
```

For:

B, A, B, C, A

expect:

B, A, C

The set answers "seen?" efficiently; the list preserves output order.

Missing key versus key mapped to missing value

These are different states:

```
record = {"sales_rep": None}
print("sales_rep" in record)      # True
print(record.get("sales_rep"))   # None
print(record.get("manager"))     # None
```

Both `.get(...)` calls return `None`, but one key exists with a missing value and the other key does not exist. If that distinction matters, test membership explicitly:

```
if "manager" not in record:
    ...
```

Do not let convenience methods erase the difference between "field absent" and "field present but unknown."

Composite keys

A tuple can be used as a dictionary/set key when identity requires multiple fields:

```
seen = set()
key = ("0101", 2)
seen.add(key)
```

This mirrors the composite-key thinking from SQL: `(order_id, line_no)` can identify an order line when `order_id` alone is not unique at line grain.

Independent task

Implement `frequency_map` and `unique_in_order` in `M04_Coding_Drills.py`. Run visible tests. Add your own tests for empty input, one repeated value and values appearing in a different order. Then write a three-record example showing the difference between a missing dictionary key and a present key whose value is `None`.

Changed retry after review

Write `duplicate_keys(records)` where each record has `order_id` and `line_no`. Return composite keys that appear more than once, without reporting the same duplicate key multiple times. Test no duplicates, one duplicate pair and two different duplicate pairs. Explain why checking only `order_id` would incorrectly flag legitimate multiple lines.

Revision in your own words

Dicts map keys to values and are natural for counts/lookups. Sets are natural for membership/uniqueness. Combine set + list when you need fast "seen" checks and preserved first-occurrence order. Missing key and key->None are different. Composite tuple keys can represent multi-field identity.

DE04-15

Stack, queue, search and sort basics

Watch first

[1. freeCodeCamp - Data Structures and Algorithms in Python](#)

REQUIRED 01:24:57-01:31:40 - Binary vs Linear Search; OPTIONAL 05:16:47-06:48:53 - Sorting Algorithms and Divide & Conquer

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

DSA-lite for the work and the screen

You do not need a computer-science degree to understand the handful of structures and algorithms that appear repeatedly in basic coding rounds and engineering discussions. This lesson covers stack, queue, linear search, binary search and sorting at a level you can implement and explain.

[Instructor demonstration: freeCodeCamp - Data Structures and Algorithms in Python:](#)

01:24:57-01:31:40 (Binary Search vs Linear Search). Sorting basics are taught directly in this book. If you want a deeper optional sorting class, the same source has a bounded chapter at 05:16:47-06:48:53 (Sorting Algorithms and Divide & Conquer); it is not required for M04. Do not watch the entire 12-hour course.

Stack: last in, first out

Picture trays stacked on top of each other. The last tray placed is the first removed.

```
stack = []
stack.append("extract")
stack.append("transform")
stack.append("load")
last = stack.pop()
print(last)
print(stack)
```

Expected last is load; remaining stack is extract, transform.

Stacks show up in undo-style reasoning, nested parsing and traversal. You do not need to force a stack into ordinary pipeline scheduling just because you learned it.

Queue: first in, first out

Use collections.deque for efficient operations at both ends:

```
from collections import deque

queue = deque()
queue.append("job-1")
queue.append("job-2")
first = queue.popleft()
print(first)
```

Expected first item is job-1. A queue models work processed in arrival order when FIFO is the actual contract.

Linear search

```
def linear_search(values, target):
    for index, value in enumerate(values):
        if value == target:
            return index
    return -1
```

This works on arbitrary order. Worst-case it inspects all n values $\rightarrow O(n)$.

Binary search requires sorted input

Binary search repeatedly discards half of a sorted search interval. If the input is unsorted, the elimination logic is invalid.

```
def binary_search(values, target):
    low = 0
    high = len(values) - 1
    while low <= high:
        mid = (low + high) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

For sorted input, search growth is $O(\log n)$. But if you must sort fresh unsorted data just once solely to search one target, sorting cost can dominate. Algorithm choice depends on the broader workflow, not only lookup complexity.

`sorted()` versus `.sort()`

```
values = [7, 2, 9, 1, 5]
new_values = sorted(values)
print(values)
print(new_values)
```

`sorted(values)` returns a new list and leaves the original unchanged.

```
values.sort()
```

sorts the list in place and returns None. If source order is meaningful evidence, copying/sorted output may be safer. If mutating the working list is intended, `.sort()` can be appropriate. State which contract you need.

Independent task

Complete the DE04-15 functions in `M04_Coding_Drills.py`: stack, queue, linear search and binary search. Sort `[7,2,9,1,5]` without mutating the original and prove both lists. Run visible tests. Then test binary search on a correctly sorted list and write a note explaining why you must not silently apply it to arbitrary unsorted input.

Changed retry after review

Use tasks ['ingest','validate','publish'] to contrast stack and queue removal order. Search target 12 inside [4,12,7,3] with linear search. Create a sorted copy before any binary-search attempt, and explain the extra sorting step. Add an absent-target test.

Revision in your own words

Stack = LIFO. Queue = FIFO. Linear search works on arbitrary order and is $O(n)$. Binary search needs sorted input and is $O(\log n)$ lookup. Sorting commonly costs $O(n \log n)$. `sorted()` returns a new list; `.sort()` mutates. State preconditions and mutation behavior when explaining an algorithm.

DE04-16

Integrated Python checkpoint and reproducible script habits

Watch first

[1. Data structures and algorithms in Python - beginner course](#)

Optional visual - no verified bounded range. Use the lists, strings and dictionary demonstrations as preparation. Save your independent checkpoint attempt before returning to worked solutions.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Syntax becomes engineering only when another person can rerun it

The final M04 checkpoint combines the module into a small data program. The goal is not to build an enterprise framework. It is to demonstrate explicit input contract, safe path handling, parsing and validation, transformation, filtering, aggregation, deterministic output, a visible failure, and a documented run command.

This is a major independent checkpoint. Do not open the Explained Review before saving [your first attempt](#). [The official Python Packaging User Guide - Packaging Python Projects is a](#) post-attempt reference only for this lesson. It currently shows a modern project shape using README.md, pyproject.toml, src/ and tests/. You are not required to publish a package or copy that structure into this checkpoint. Open it only after your attempt to see how the same reproducibility ideas grow in professional Python projects.

Picture a tiny data factory

```
input path
-> file exists?
-> required columns?
-> parse/validate values
-> calculate sales
-> filter Completed
-> aggregate / top-N / missing count
-> deterministic JSON
-> read-back validation
-> saved run evidence
```

Each arrow is a boundary where an assumption can be tested.

Your input contract

For data/M04_Retail_Orders.csv, required columns are:

```
order_id, region, status, qty, unit_price, sales_rep
```

One source row represents one order in this training fixture. order_id must be unique. qty and unit_price must be parseable numeric values appropriate for the calculation. Do not silently replace malformed values with zero. Raw input remains unchanged.

Your output contract

Produce one deterministic JSON document containing at least:

- source filename;
- source row count;
- Completed row count;
- Completed sales total;
- regional Completed totals;
- top 3 Completed orders under a declared deterministic sort;
- missing sales_rep count over a clearly named population.

Stable output means the same valid input and code produce the same logical structure and ordering. Use a deliberate sort order for dictionaries/lists that otherwise depend on source order or equal values. `json.dump(..., sort_keys=True, indent=2)` helps stable text formatting but does not by itself define how your top-3 list handles ties; you must define that in the data logic.

Validation must challenge a plausible wrong result

At minimum prove:

- input rows = 25;
- Completed rows = 23;
- Completed sales = 39320;
- regional Completed totals reconcile to 39320;
- one missing sales rep is visible under the chosen population;
- top-3 order IDs/values satisfy your sort rule;
- raw CSV hash/content is not modified by your program;
- output JSON can be read back and critical values match what was calculated.

Do not paste these numbers into the output as constants. They are controls to compare against the values your pipeline calculates.

Deliberate failure proof

Run the program with `data/M04_Bad_Orders.csv`. The supplied bad file lacks required qty. Your checkpoint should fail at the schema boundary with a clear message rather than stumbling later into a `KeyError` or fabricated calculation.

Also test a missing file path. Those are two different failure boundaries: path existence versus schema contract.

Reproducibility evidence

Write an exact command such as:

py M04_Checkpoint_TODO.py

or whatever explicit argument form you designed. Close/reopen the terminal and prove the documented command still works from the location you documented. Another learner should not need your personal desktop path or editor button.

Your evidence bundle should include first attempt, observed controls, output JSON, one bad-input failure, run command/transcript and a short architecture explanation.

Teach it back as if an interviewer asked

Without notes, explain:

1. What is the input grain?
2. Where is schema validated?
3. Where are numeric values parsed?
4. Why is raw input not overwritten?
5. What population is used for each metric?
6. How do regional totals reconcile?
7. What makes top-3 deterministic?
8. What failure did you prove?
9. How can another person rerun the script?
10. What would you improve before calling it production software?

A strong answer can admit limits. M04 is foundation Python. Logging, reusable package design, tests, configuration and pipeline robustness are deepened in M05/M06.

Changed full checkpoint after review

Only if the review/repair route assigns it, close all answer material and implement

M04_Fresh_Checkpoint_TODO.py against M04_Retest_Shipments.csv. This changed scenario

uses shipment/hub language, Delivered filtering and different controls. It is not a word-for-word clone of retail orders.

You must validate schema; calculate shipment_value; aggregate Delivered value by hub; produce deterministic top 3; count missing coordinator values over a declared population; write/read JSON; prove one failure; and explain input/report grain. The fresh review remains closed until that attempt is saved.

Revision in your own words

A reproducible script has an explicit entry point, input contract, validation boundaries, deterministic output and documented run command. Validate before transformation, preserve raw input, reconcile important totals, prove a failure path and read outputs back. "It ran on my laptop" is not the same as reproducible engineering.

M04 move-on rule

Do not move to M05 because you read all pages. Move when all lesson evidence is complete, any reviewed weakness has a successful changed retry, the four cluster checks are satisfactory, and the DE04-16 checkpoint has no unresolved critical defect. G03 is still later: M04 + M05 + M06 feed that protected Gate.

Revision rhythm

Use task-based revision at D+1, D+4, D+10 and D+30. From DE04-13 onward keep the 15-20 minute Python/DSA-lite micro-drill alive. Do not reread the entire learner book for revision unless you genuinely cannot reconstruct the concept; use the ready-made revision lines, then solve a fresh task.